

A Empirical Study of 15 Android BLE Health Applications: Comprehensive Findings Report

Introduction

Bluetooth Low Energy (BLE) health and fitness applications form a fast-growing category on Android. A typical app in this class connects to a bonded peripheral (a blood-pressure cuff, a glucometer, a body-fat scale, a fetal-heart monitor, a continuous glucose monitor, or a wearable), reads a stream of physiological measurements over GATT, persists those measurements together with identity and device metadata on the phone, and synchronises them to a vendor cloud or to a third-party health platform. The same app is also the place where the user types in profile data such as name, date of birth, height, and weight, where the access token to the vendor backend is stored, and where third-party analytics, push, and crash-reporting SDKs are initialised.

A single BLE health app therefore exposes three interacting attack surfaces: the BLE link itself, the Android application layer, and the network / cloud layer. Most published audits look at one of these surfaces in isolation. This study takes the opposite view. It applies a single rule-driven static workflow to fifteen real Android BLE health applications and examines, app by app and surface by surface, what conclusions are actually supportable from the shipped code.

The results are reported as category-level empirical roll-ups rather than isolated case studies. For each rule category, we list the affected applications, assign an evidence tier, and expand representative cases with source-level evidence

1. Background

1.1 Why BLE health apps are a distinct target class

Compared with general-purpose Android apps, BLE health applications combine three properties that change their security posture. First, the data density per user is high: a single account routinely carries continuous physiological measurements (blood pressure, blood glucose, oxygen saturation, heart rate, body fat, fetal-heart waveforms, training sessions) plus stable identifiers (account id, device serial, MAC address) plus profile data (name, phone, age, height, weight). Second, the engineering posture around transport encryption is, on average, weaker than in financial or communication apps; many of the apps reviewed here still ship with cleartext traffic permitted at the package level, with mixed HTTP and HTTPS base URLs in shipped assets, or with TLS-bypass code paths on real product flows. Third, the consequence of a leak is more severe, because the leaked record is medical and is paired with full identity. These three properties also make the issues easier to surface from a static review.

1.2 The three layers in scope

The review treats every app under three layers. The BLE link covers GATT scan, connect, characteristic read/write/notify, command framing, and pairing/bonding posture. The Android application layer covers Manifest declarations and exported components, local persistence (SharedPreferences, Room/ObjectBox/SQLite, custom files), crypto, logging, WebView and JavaScript-bridge configuration, third-party SDK initialisation, and the dynamic flags visible in release builds (debug toggles, log strings, build configuration). The network / cloud layer covers HTTP and HTTPS endpoints, the TLS configuration of the clients used to reach them, the authorisation material carried in headers and bodies, and the destinations of third-party telemetry.

1.3 Evaluation criteria (two tiers)

Every finding in this report carries one of two evidence labels. A finding labelled confirmed is observable directly in the shipped artifacts (Manifest, Smali / Java sources, decompiled assets, embedded XML / JSON), and the underlying evidence file lists the exact source file and line number. A finding labelled supported_hypothesis is one whose static chain is closed enough that the risk is consistent and plausible, but whose final empirical step (a captured packet, a real device exchange, an actual exfiltrated byte) was not produced in this study.

2. Methodology

2.1 Application selection

Fifteen Android applications were selected. Two are general BLE explorers and SDK demos: **nRF Toolbox** and **LightBlue**. Twelve are vendor-branded health or fitness apps: **BLE Health Plus**, **bruHealth**, **Bupa Anywhere**, **FitPro**, **Yuwell HealthManager**, **MyVitale**, **Nutrack**, **Polar Beat**, **Rossmax Connect**, **WeightTrack**, **Senssun SmartLink**, and **Yuwell uHealth**. One is an open-source diabetes monitor: **xDrip+**. Each app was required to contain a real BLE pairing or measurement code path in the shipped APK, to be available for download, and to be in current distribution at the time of the review.

2.2 Two literature databases

Two distinct sets of papers were used and they play different roles in the workflow. The first, called database_original, was assembled in the early phase of the project before the workflow was settled. It is a relatively unfiltered collection of papers covering possible privacy and security problems in mobile health and IoT applications. Its purpose was to build personal background knowledge of the threat landscape. The free-form notes in finding.txt files were written from this material together with my own intuition and prior knowledge, and they are therefore the more subjective layer of the project record.

The second, called database_main, was assembled later once the workflow was being formalised. Each paper in database_main was selected because its findings are concrete enough to be expressed as a discrete rule. Each rule was then manually translated into a small set of search keywords (Java identifiers, Manifest attributes, characteristic UUIDs, crypto suite names, file-system patterns) consumed by Python scripts that scan the decompiled APK with ripgrep. Every machine hit was then verified by hand against the source line. The summary_notes.txt file produced for each app is the output of that path: every issue listed in summary_notes.txt maps back to a specific rule and to a specific paper in database_main. This report draws primarily from that material.

2.3 Per-application pipeline

Each APK was unpacked and processed under the same pipeline. The package was decoded with apktool to recover the Manifest and resources, and decompiled with JADX to recover Java sources; assets such as embedded HTML, JavaScript, JSON configuration, and SQL migrations were extracted directly. The rule keywords were then run as Python ripgrep searches across the decoded tree, and every hit was opened, read in context, and either confirmed against the rule or rejected. A finding was

categorised under the rule whose keyword produced the hit, and assigned an evidence tier of confirmed or supported_hypothesis according to the criteria in section 1.3.

3. Results

The results are organised as category roll-ups. Each subsection opens with a short pattern description, lists all apps in which the issue appears with an evidence tier, and then expands a small number of representative cases with file-and-line evidence.

3.1 BLE link layer

Across the vendor-branded apps the BLE code path is structurally similar: scan for advertisements that match a vendor service UUID, connect with GATT, enable a notify characteristic, decode the incoming bytes into a measurement object (blood pressure, glucose, SpO2, weight, heart rate, fetal-heart audio frame, training session), and forward that object to local storage and to the upper layer. The reverse direction — commands such as history sync, time set, Wi-Fi configuration, Wi-Fi list query, OTA, device reset, lock / unlock, payment-QR push — is also present in several apps.

Confirmed: the GATT characteristic and command surface is exposed and reachable from a first-party caller in every vendor app: **BLE Health Plus** (BaseBleActor + BleConnectionService), **bruHealth** (GlucometerBridge), **FitPro** (BaseLeService + BaseReceiveData + SendData), **Yuwell HealthManager** (BPMManager, GlucoseManager, OximeterManager, ThermometerManager, FetusManager), **MyVital** (OmronSessionManager + SessionController), **Nutrack** (classic-Bluetooth RFCOMM SPP and Wi-Fi TCP sockets — not GATT — with Balanza control commands ":L", ":M", ":N"), **Polar Beat** (PFTP protocol on fixed server UUIDs and MTU, BR063/066 confirmed), **Rossmax** (VUBleManager), **WeightTrack** (no first-party BLE path, excluded from this category), **xDrip+** (DexCollectionService + transmitter characteristics), **Senssun SmartLink** (GATT scan/connect/notify with command helper), **Yuwell uHealth** (MoxManager + BLE callbacks), **nRF Toolbox** and **LightBlue** (general explorer surfaces).

Supported_hypothesis: application-layer authentication of the peer (HMAC, signature, challenge-response, per-session keying, device certificate check, replay defence) is not visible in the first-party code of any of these apps. The attack-style rules (BR061–BR075, rule49–55 / 61 / 63, R073–R084) therefore remain supported, not confirmed, because the static review cannot prove that the real peripheral accepts spoofed, replayed, or injected frames.

Representative detail. In **Yuwell uHealth**, MoxManager and UHBleCallbacksHandler parse BP, glucose, and SpO2 measurements directly into measurement models with no application-layer HMAC or signature step before the bytes move inward (BR066 confirmed; BR061-BR065, BR067-BR069, BR073-BR075 supported_hypothesis). In **FitPro**, SendData.java:314 builds the generic protocol frame, SendData.java:294 builds pairing, :123-125 sets user information, :405-406 builds payment payloads, and BaseReceiveData.java:1798-1799 calls uploadHeartBlood(measureDetailsModel) immediately after saving the measurement — none of which is wrapped with a visible HMAC / signature / nonce layer. In **Polar Beat**, BluetoothService + clq.java:18-23 + clr.java:44-48 route writes by Bluetooth device address and characteristic UUID into a PFTPController; the PFTP protocol surface is confirmed but the attack-style rules (BR061/062/064/067/068/069/071/074/075) stay supported. In **Senssun SmartLink**, a command helper builds Wi-Fi configuration, Wi-Fi list, Wi-Fi OTA, and history-sync commands without an obvious application-layer signature gate, so the "visible app path does not show an obvious app-layer MAC, signature, or provenance proof" conclusion is explicitly retained in rule49 / 50 / 51 / 53 / 54 / 55 / 61 / 63.

Nutrack is a special case. Its device link is classic Bluetooth RFCOMM plus Wi-Fi TCP, not GATT: Bluetooth.java:7-8 fixes the SPP UUID and port 5373, BluetoothClient.java:54 and BluetoothServer.java:45 use insecure RFCOMM, WiFiClient.java:35,54 and WiFiServer.java:28,61 use

ordinary sockets, ProtoFeedMsg.java:92-115,139-143 uses only unkeyed MD5 integrity, ProtoFeed.java:314-350,520-576 sends STX + JSON + ETX frames. The attack surface is adjacent to the BLE category rather than inside it, but the application-layer gap — no HMAC, no signature, only keyless MD5 — is the same.

3.2 Application reverse-code analysis (static and dynamic)

3.2.1 Cleartext transport policy and TLS posture

Package-level cleartext permission is one of the most consistent confirmed findings of the set.

Confirmed cleartext-permitted policy: **FitPro** (AndroidManifest.xml:154 usesCleartextTraffic="true" plus network_security_config.xml:3 and :13 both allowing cleartext, BR006/BR013 confirmed), **Nutrack** (AndroidManifest.xml:61 usesCleartextTraffic="true", BR006 confirmed), **Bupa Anywhere** (AndroidManifest.xml:125 usesCleartextTraffic="true", BR006 confirmed), **xDrip+** (AndroidManifest.xml:139 usesCleartextTraffic="true", BR006/BR012 confirmed), **Yuwell uHealth** (network_security_config.xml:3 cleartextTrafficPermitted="true", BR006/BR008 confirmed), **Senssun SmartLink** (usesCleartextTraffic="true" plus HTTP business URLs shipped in the bundle, rule06 confirmed), **Yuwell HealthManager** (global cleartext allowance kept as broader posture, supported).

Confirmed code-level TLS bypass on real product paths:

xDrip+ — ShareRest.java:98 has an empty checkServerTrusted; ShareRest.java:140-143 returns true unconditionally from the HostnameVerifier; the same client is used at ShareRest.java:174-176 for Dexcom Share authentication (account name and password) and at ShareRest.java:204-208 to upload BG records (BR009 / BR010 confirmed).

Polar Beat — bqh.java:33 builds the DefaultHttpClient; bqh.java:38 sets ALLOW_ALL_HOSTNAME_VERIFIER; bqj.java:16-17 installs a custom TrustManager; bqj.java:16,20 leaves checkClientTrusted empty and has only weak server-certificate logic; bqf.java:21,25,26 uses that client to send /register/user with username and password (BR009 / BR010 / BR012 confirmed).

bruHealth — the StepsUploader path uses an SSLSocketClient with an empty checkServerTrusted plus a HostnameVerifier that returns true, installed through OkHttpUtils.setSafe; the StepsUploader uses that client to upload step and calorie data together with wlapptoken and device-id. The main ApiClient path is weaker but also real: EvydHostnameVerifier returns true for the configured base host branch (BR009 / BR010 / BR012 confirmed).

Apps without completed evidence for this category: **nRF Toolbox**, **LightBlue**, **MyVitale**, **Rossmax**, **BLE Health Plus**, **WeightTrack**, **Yuwell HealthManager**, **Yuwell uHealth**, **Senssun SmartLink**, **FitPro**, **Nutrack**, **Bupa Anywhere** do not show a first-party trust-all TrustManager or an always-true HostnameVerifier on a business client; those TLS-bypass rules stay out for these apps.

3.2.2 Hardcoded crypto keys and weak modes

Confirmed hardcoded-key / weak-mode cryptography on real product paths:

Yuwell uHealth — EncryptUtil.java:22 hardcodes "8yu&well"; line 56 decrypts with DES/ECB/NoPadding; line 85 encrypts with DES/ECB/PKCS7Padding; SyncService.java:120,131 wraps the upload payload with that encrypt call and DatabaseServiceImpl.java:1498,1508 decrypts incoming server data with the same utility, so this is part of the live sync path and not dead code (BR017 confirmed).

xDrip+ — Backup.java:621 uses AES/CBC/PKCS5Padding with the key 5861bdb93f5ba8effa43c8ff7f26c3be hardcoded at Backup.java:623 (BR017 confirmed).

BLE Health Plus — AESCipher uses IV_STRING = "BlupixelHealthPI" and Cipher.getInstance("AES") (mode/padding not specified, implying a weak default); the same cipher wraps the full Room database that BackUpFragment checkpoints, writes as ble_health_backup.bkup under externalCacheDir, and uploads to Google Drive (R051 / R054 / R057 / R059 / R060 supported through this backup chain).

Nutrack — Encrypter.java:15-16 defines a fixed IV and a fixed AES key; lines 19-23 configure AES/CBC/PKCS5PADDING; MainActivity.java:1044-1049 reads **NutrackLicense**, decrypts it, and extracts lic_uuid (BR017 confirmed).

The boundary is identical in all four cases: the existence of weak crypto inside the client does not prove that the outer transport is missing or that the server has no other check, but it does confirm a fixed-key DES or fixed-IV AES design inside the client sync / backup chain.

3.2.3 Local persistence of credentials, tokens, and health data

Confirmed plain local persistence, categorised by what is stored:

(a) Access tokens / session credentials. **Yuwell uHealth** — PreferenceSource.java:59-64/315-317 reads and writes access_token, :209-214/476-478 stores the refresh token (BR026 confirmed). **xDrip+** — BluetoothScan.java:309/327/486/515 writes the share key, last connected address, receiver serial, and transmitter id; ShareRest.java:260 writes the Dexcom session id; AuthFlowOut.java:55 writes the Tidepool AuthState JSON; MiBand.java:127 writes the MiBand authentication key (BR026 confirmed). **Polar Beat** — UserData2 SharedPreferences hold username, password, profile_json, address_json, and current_device_id; BeatPrefs.java:331/333/420/454/573/608/732 enables autoSignPossible and reuses the saved credential fields (BR026 confirmed). **WeightTrack** — worktrack_setting SharedPreferences hold Fitbit access tokens and Withings token/secret material (BR026 confirmed). **bruHealth** — Room user_info with user_id/member_id/id_no/id_type/phone_no/region_code, SPUtills Constants.USER_TOKEN, PersistentCookieStore writes serialised cookies into CookiePrefsFile (BR025 / BR026 confirmed). **Bupa Anywhere** — index.android.bundle:734 uses redux-persist-transform-encrypt whose secretKey is produced by getKey() inside the same packaged bundle, and the persist whitelist includes session, monitor, resources, and shared (authToken, authTokenExpires, idToken, refreshToken) (BR026 / BR016 confirmed). **MyVitale** — SessionManagerSP.java:12/21/32 opens session_local_store and saves the login token; PreferenceManager.java:12-13/27-41 reads and writes partnerKey and measuresServerUrl (supported). **Senssun SmartLink** — the Flutter layer persists a login credential entry and an account info entry, and flutter.sp_token is then read again by the native side (rule26 confirmed). **LightBlue** — app_preferences writes ADAFRUIT_IO_USERNAME and ADAFRUIT_IO_SECRET_KEY without an encryption wrapper (supported).

(b) Health measurements and profile records. **BLE Health Plus** — AppDatabase_Impl defines BloodPressureTBL, HeartRateTBL, GlucoseTbl, WeightScaleTBL with user_id / device_mac / date_time / raw data (R006 confirmed). **FitPro** — DBModule.java:32 sets ENCRYPTED = false and the schema covers health, identity, token, password, device, and location fields (BR025 confirmed). **Yuwell HealthManager** — ObjectBox initialised without an encryption key; DataRepository / BaseDAO use BP, BG, OXY, HTS, and FetalHeart measurement boxes (R051 / R054 confirmed). **Yuwell uHealth** — uhealth.db with BPMeasurement / BGMeasurement / OxyData / FamilyMember models (BR025 / BR018 confirmed). **xDrip+** — BgReadings with calculated / filtered / raw / sensor / source / timestamp fields, Treatments with carbs / insulin / notes / insulinJSON, BloodTest, Sensor with sensor location, plus on-disk migrations under assets/migrations/ (BR025 confirmed). **Polar Beat** — SQLite / Sugar ORM ProtoEntity, UserPhysicalInformation, TrainingSession (TrainingSession.java:611/612/696/710/716) and Exercise (route, RR samples, samples, statistics, transition at Exercise.java:720-764) (BR025 / BR018 confirmed). **WeightTrack** — ActiveAndroid WTA.mp3 with Weight, Goal, and Image models covering weight, BMI, BMR, fat, muscle, water, heart rate, waist, hip, chest, bicep, thigh, diary, calorie, age, height,

sex, target weight (BR018 / BR025 confirmed). **Senssun SmartLink** — smartlink_v1 with user_v2, device_v1, bodyFatScaleRecords_v5, bloodPressure_v1, bloodSugar_v1, bloodUricAcid_v1, heartRate_v1 (rule25 confirmed). **Nutrack** — **Nutrack.db** with usuarios(username, password) at **NutrackDbHelper.java:186**, implementos with Bluetooth / Wi-Fi address at :188, corrales with geolocation at :196, plus production / task / feeding tables at :210-222 (BR025 confirmed).

(c) User macros and other custom payloads. **nRF Toolbox** — a SQLite table configurations(_id, name, xml, deleted) stores user-defined UART macros whose XmlMacro.command field is free-form text; the macro may be harmless, but may also hold device secrets or control payloads (BR025 supported_hypothesis).

3.2.4 External-storage export and backup exposure

Confirmed wide backup / export policy paired with sensitive local state:

FitPro — backup_rules.xml contains only <full-backup-content/> and data_extraction_rules.xml permits <cloud-backup/>; no exclusions were added for the sensitive DB or session material (BR028 confirmed); the DB also sits under an external app-specific path via PathUtils.java:30/101-102, and WebSocket voice AMR files land on external app music storage (BR027 confirmed).

WeightTrack — allowBackup="true" and the app exports WTA.mp3 as **/WeightTrack/WTa.data**, also writes weight_backup.json and CSV, and shares the file with ACTION_SEND (BR027 / BR028 confirmed).

Nutrack — allowBackup="true" and no exclusion rules for **Nutrack.db** or the sensitive preference entries (BR028 confirmed).

Senssun SmartLink — allowBackup="true" alongside the smartlink_v1 local store containing token, account, device, and measurement state (rule28 confirmed).

BLE Health Plus — the backup path uses the AESCipher of 3.2.2 to write ble_health_backup.bkup under externalCacheDir and then uploads it to Google Drive (R051 / R054 / R057 / R059 / R060 supported).

Yuwell HealthManager — allowBackup=true plus local fetal raw/audio files, Excel exports, and fetal screenshots (supported broader posture).

MyVitale — AndroidManifest.xml:30 enables backup and the app stores access tokens plus configuration in ordinary SharedPreferences (supported).

Cleared for this category: **Bupa Anywhere** (allowBackup="false"), **nRF Toolbox** (allowBackup="false" with data_extraction_rules excluding sensitive areas), **xDrip+** (allowBackup=false).

3.2.5 Release-build logging of sensitive data

Logging is not a single debug-line concern in this set; it is repeatedly a complete chain that takes sensitive sources into log surfaces that are still active in release builds.

Confirmed release-build logging of sensitive values:

MyVitale — BuildConfig.java:6-7 shows DEBUG=false; AppLog.java:7-8 still enables output logging at verbose; SessionController.java:192-193 logs the device address and the session-option map at session start; :308 logs UserData; :323 logs MeasurementRecords; OmronSessionManager.java:179 prints the latest measurement record set; MeasuresViewerPresenter.java:231-253 prints weight, BMI, basal metabolism, body fat, body water, fat-free mass, muscle mass, visceral fat, and skeletal muscle (R050 confirmed).

Yuwell HealthManager — Constants.java:17-19 defines the fetal raw / audio / screenshot paths; FetalMeasureActivity.java:145-146 opens the raw and audio files, :154-155 writes bytesAudioData and rawData, :252 creates FetalHeartMeasurement, :256-257 stores the local raw and audio file paths; FetusHome.java:60 logs the recent fetal result; Ret.java:27 includes the data object in toString(); FetalHeartMeasurement.java:19 and :26-30 include value, measure time, and local file paths in toString() (R050 confirmed).

Yuwell uHealth — SyncService.java:69-70 logs decrypted sync data; SyncService.java:114-125 logs the upload JSON together with UID, entityKey, uploadData, accessToken, deviceId; UHBleCallbacksHandler.java:197 logs the BLE address; :274 logs glucose JSON; MoxManager.java:44/56/232 logs received bytes, sent bytes, and parsed data; Logger.java:55 wires AndroidPrinter and FilePrinter under an external-cache LOG directory; LogUploadUtil.java:28-49 compresses and uploads those logs to a server (BR047 / BR027 confirmed).

xDrip+ — ShareRest.java:120 logs request bodies and :128 logs response bodies on the same path that exchanges Dexcom credentials and BG values (BR047 confirmed).

Senssun SmartLink — MainActivity.initData() prints flutter.sp_token through Log.i (rule41 confirmed).

FitPro — MineFragment.java:281-282 logs token, Bluetooth address, and BLE state together; JWebSocketClientService.java:45 logs inbound WebSocket messages; UploadDataService.java:37-41 logs upload start plus device IDs; BaseLeService.java:498 writes raw BLE bytes to local logs; BaseReceiveData.java:876-879/1436/1466/1648-1689 logs parsed step, movement, and sleep values (BR047 confirmed).

WeightTrack — WithingsActivity prints OAuth token responses and measurement responses with System.out.println(strString) (BR047 confirmed).

bruHealth — sensitive values reach log sinks (health and route upload parameters, request headers, authentication context, location data); the product-file runtime output is kept as a separate unproven step because of WorkInitialization / EvydEnvWrapper.isProd() gating (BR047 confirmed at the sink level).

Supported logging exposure (sink visible, no captured logcat): **BLE Health Plus** (ApiController.getHttpClient enables HttpLoggingInterceptor.Level.BODY across login, registration, OTP, profile, subscription, and image upload APIs; AppLog is empty and is not treated as evidence); **nRF Toolbox** (NativeGattCallback prints characteristic UUID and value; ProfileService creates nRFLoggerTree with deviceAddress); **LightBlue** (K5.b.java:137-141 logs scan discoveries with device name and MAC, J5.C1754d.java:216-217/294/332/342/1314/1572 logs bond state, connection, write, read, and changed characteristic values; Q5.d.java:211 renders log events with value bytes as a string); **Rossmax** (login logs Google and Firebase fields, Retrofit BASIC logging, some Cloud Functions URLs carry access_token in the query string, ViTree handles Timber.i).

3.2.6 Exported components and WebView / JS bridges

Exported components and WebView entry points add a controllable surface that an external caller can reach.

Confirmed exported first-party components with sensitive downstream behaviour:

Yuwell uHealth — AndroidManifest.xml:275-284 exposes UHealthWebViewActivity with the action com.yuwell.uhealth.action.START_ACTIVITY; UHealthWebViewActivity.java:47 reads the title extra, :63 reads the URL extra and loads it; the WebViewClient at line 29 follows redirected URLs (BR029 / BR059 confirmed; BR035 supported).

FitPro — AndroidManifest.xml:219-221/235-236/244-245/259-260/292-293/353-354/396-397/607-609 exposes JWebSocketClientService, BluetoothCommandActivity, DebugFunctionActivity, ECGHistoryActivity, PayQrcodeActivity, HealthReportActivity, ApiDebugActivity, and UploadDataService;

DebugFunctionActivity.java:73-75 sets Constant.isDeubg = true and at :224/:228 launches ApiDebugActivity and BluetoothCommandActivity; BluetoothCommandActivity.java:33 reads weather.json and :83-99 writes weather bytes into commandPoolWrite; PayQrcodeActivity.java:85 reads the PAY_EXTRA intent extra and :142-144 sends the QR content into SendData.getPayValue; ECGHistoryActivity.java:31-35 reads ECG history with DBHelper.queryECGData; UploadDataService.java:35-42 begins uploading as soon as it is created (BR029 / BR030 / BR059 confirmed).

Bupa Anywhere — AndroidManifest.xml:132-162 exposes RNBootSplashActivity; :163-234 exposes MainActivity; RNBootSplashActivity.java:14-22 copies extras, data, action, and type into the next intent; MainActivity.java:90/94-103/106-109/113-119/149/158-159/178 reads the launch type, handles ACCEPTED / INCOMING_CALL / STOPPED, and wires RNBranchModule.initSession(getIntent().getData()); the service layer closes the chain through RingerService.java:26-29/53-58/66-71/157-170/174-185 and OngoingCallService.java:32-35/49-50 (BR029 / BR059 confirmed).

Supported exported-component exposure (sink not fully closed or gated by other checks): **BLE Health Plus** (BleScanService exported without caller validation; ReminderReceiver exported, processes incoming action + reminder data, rebroadcasts the reminder event; R017-R020 supported). **Yuwell HealthManager** (many exported statistics / measurement / result / WebView activities at AndroidManifest.xml:51-216 with intent-extra reads in BgResult / BpResult / HtsResult / FhtResultActivity; R017 / R018 supported). **xDrip+** (SyncService, ExternalStatusBroadcastReceiver, InfoContentProvider exposed — provider query path incomplete in decompilation; BR030 / BR031 / BR021 / BR024 supported). **Senssun SmartLink** (JPush DownloadProvider without protection; rule24 supported).

Confirmed WebView / JavaScript-bridge surfaces carrying identity or health data:

BLE Health Plus — ShareReportFragment.loadWebView() builds a ReportModel, serialises it with Gson, and injects it with evaluateJavascript("javascript:loadData('...')", null); the model carries name, email, age, gender, profileUrl, deviceType, readingData, chart values, and settings; assets/report.html loads remote scripts from Google CDN, Highcharts, and rawgit.com (R033 / R034 confirmed; R030 / R035 supported).

Yuwell HealthManager — BgStatistic.java:168/179/218-219/268, BpStatistic.java:159/170/209-210/254, BoStatistic.java:99/166/172-173, and HtsStatistic.java:120/187/193-194 inject real health statistics into WebView JavaScript, enable JS and DOM storage, and call setWebContentsDebuggingEnabled(true) in release (R034 supported; the release-build debuggable WebView is the dynamic flag on these four pages).

Rossmax — WebFragment enables JavaScript and automatic window opening for remote content; SetRegionActivity enables JavaScript and uses onJsPrompt(...) without validating the url origin before writing regionShortCode into Firestore after user confirmation (R029 / R031 supported).

bruHealth — CommonWebView auto-grants geolocation to any origin; the H5 / Flutter bridge can return app token and user info if ever exposed to untrusted content; setMixedContentMode(0) is also set (independent engineering finding; kept visible alongside the rule catalogue).

Senssun SmartLink — the deep-link ext → WxUtils → JavaScript callback state flow reaches JS state; rule29 / rule34 / rule35 / rule71 / rule72 supported (the chain reaches JS state but does not yet reach a proved final sensitive sink).

3.2.7 Deep links and custom schemes

Confirmed browsable / deep-link entry points:

Bupa Anywhere — bupabluahealth scheme plus account host, plus MainActivity callback filters (BR032 confirmed; BR033 / BR034 supported; BR035 / BR084 not_testable).

bruHealth — DeepLinkActivity exported and browsable on the bruhealth scheme, routing into SchemeOpenManager (BR032 confirmed; BR034 / BR035 supported).

WeightTrack — multiple wta:// deep links; Fitbit and Withings callback links pass the code parameter into token-exchange logic; wta://measure opens MeasureActivity, which reads PARAM_CATE and shows measurement data without going through the pattern-lock flow that lives in NavigatorActivity (BR032 / BR034 / BR035 confirmed; BR029 supported).

Polar Beat — oauth://fi.polar.Twitter_oauth_Settings (BR032 confirmed; combined with a targetSdk of 22 which changes the exported-component default behaviour).

Rossmax Connect — https://r.rossmax.care App Link with autoVerify and no path restriction; SplashActivity forwards inbound data into MainAdvActivity; DeepLinkUtil.resolveDeepLink() reads the invitation id (R026 / R028 supported; non-anonymous-user checks, expiration checks, used checks, and a follow confirmation dialog are also visible, which is why the rules stay below confirmed).

Senssun SmartLink — smartlink:// MainActivity browsable entry; first-party code reads getIntent().getData() (rule32 confirmed; later deep-link rules rule29 / 34 / 35 / 71 / 72 supported because the chain reaches the JS ext state but not a final sensitive sink).

xDrip+ — xdrip://callback/tidepool RedirectUriReceiverActivity uses AppAuth with PKCE S256 at AuthFlowOut.java:113-114; BR032 / BR035 stay supported_hypothesis exactly because the protective logic is visible too, which is why BR033 / BR034 are cleared.

3.2.8 Third-party SDKs and the identifier surface

Third-party SDKs are responsible for the bulk of the identifier read surface, and the confirmed / supported split is driven by whether the static evidence reaches only the initialisation or also the actual egress bytes.

Confirmed static SDK initialisation and extractable key material:

Bupa Anywhere — appcenter-config.json:2 carries the AppCenter secret; strings.xml:175-176 carries Braze/Appboy key material; strings.xml:276,278 carries Google key material; AndroidManifest.xml:280-284 carries Branch key material; index.android.bundle exposes the Sentry DSN and liveChat-style keys; MainApplication.java:113-121 initialises Braze, AppCenter, and Branch; AndroidManifest.xml:67 declares AD_ID and :98 declares READ_PHONE_STATE; index.android.bundle:859 shows authorisation-parameter construction with device-derived and visitor-style values (BR014 / BR016 / BR040 confirmed; BR041 / BR092 supported).

FitPro — Constant.java:70 hardcodes Bearer 6fcb7f58475b4e5aad8f0f1cadce235e; DBHelper.java:304-323 provides the fallback-to-hardcoded-token lookup; HttpHelper.java:683-692 uses Constant.TOKEN for updateDeviceInfo; strings.xml:1612-1613 / 2295-2296 / 2391 / 2659 / 2783 holds QQ, Sina, Tencent Auth, Umeng, and WeChat app-id / secret material (BR016 / BR015 confirmed; BR040 / BR041 / BR052 / BR092 supported).

LightBlue — O5.a.java:137 initialises Mailchimp with a packaged API key; X5.C4736b.java:34 builds the Mailchimp endpoint; InterfaceC4737c.java defines contacts / events endpoints; App.java:99-107 runs on startup and enables Firebase Analytics and Crashlytics when Firebase apps exist; preferences.xml:4-7 sets send_usage_data default to true; AndroidManifest.xml:29 / :44 requests ad-related identifier permissions (BR016 confirmed; BR026 / BR047 / BR061 / BR067 / BR068 / BR075 / BR091 / BR092 / BR097 / BR098 supported).

nRF Toolbox — strings.xml contains google_api_key / google_app_id / project_id; FirebaseInitProvider, Crashlytics, Sessions, Installations, DataTransport are registered; NrfToolboxApplication.java queues APP_OPEN and wb.c.java persists the user choice and controls Firebase Analytics collection state. The

Manifest defaults firebase_analytics_collection_enabled=false, google_analytics_adid_collection_enabled=false, and google_analytics_ssaid_collection_enabled=false are visible gating, which is why BR041 is explicitly not_supported for this app (BR016 confirmed; BR040 / BR092 supported).

Senssun SmartLink — third-party namespaces (JPush, UMeng) read TelephonyManager.getDeviceId and getSubscriberId; Build.BRAND, Build.MODEL, locale, display metrics, and AD_ID are collected by the same third-party code; rule02 / rule03 / rule14 / rule15 confirmed for the third-party side. MyApp.onCreate() calls UMConfigure.preInit before the user-agreement gate, but the same path also sets JCollectionAuth.setAuth(false) before consent and turns JPush on only after the agreement is accepted — the consent picture is mixed, which is why rule36 / rule37 stay supported.

Yuwell uHealth — GlobalContext initialises JPush, Udesk, UMeng, and Youzan; Startup.java:157-158 initialises and preloads Youzan; Manifest exposes JPush, OPPO, and mPaaS configuration values (BR040 / BR016 confirmed; BR041 supported).

Polar Beat — Facebook and Google tracking; bgn.java:538 writes advertiser_id into JSON; bdm.java:61 uses reflection to access AdvertisingIdClient.getAdvertisingIdInfo (BR040 confirmed; BR092 supported).

Supported third-party surface only: **BLE Health Plus** (Firebase Messaging / Analytics / Crashlytics / Installations / DataTransport + FirebaseInitProvider registered; LaunchActivity.onResume fetches the FCM token and stores it locally; R002 / R041 / R065 supported); **MyVitale** (Omron SDK ServerInfo.java:10 contains https://%s.firebaseio.com/bqInsert/%s/%s; BLEErrorInfo.java:61-64 calls sendBigQuery → BigQueryREST.insert; BigQueryREST.java:31-41 generates and stores a UUID and :45-65 adds common device info, formats JSON, stores cache, and posts a runnable; R004 / R052 / R072 supported); **Rossmax** (Facebook SDK auto-init + Firebase Analytics; FireAnalytics logs vital_unit / BP / BG / weight / temperature categories; R041 / R062 / R065 / R071 supported); **Nutrack** (Crashlytics setCustomKey embeds usuario, android_id, cuenta, licencia; the ad-identifier rules stay supported because there is no captured payload; BR092 supported); **WeightTrack** (BR001 / BR002 / BR003 / BR092 supported).

Device identifier reads observed directly in first-party code:

FitPro — CommonService.java:322 defines updateDeviceInfo(...) with mac, deviceModel, deviceVersion, appVersion, osVersion, osType, osLang, phoneModel, userId; HttpHelper.java:683-692 builds that call from local device data (BR015 confirmed).

Nutrack — MainActivity.java:1042 reads android_id and the same activity also writes it into Crashlytics custom key logic; **NutrackDbHttp**.java:600 reads android_id again and :670 attaches it as dev_id in the SYNC.php query string; :687-690/694 builds the POST request with Bearer authorisation (BR014 confirmed).

Senssun SmartLink — AD_ID in the package; third-party SDK code reads IMEI / subscriber id / device id / Build.BRAND / Build.MODEL / locale / display metrics (rule02 / rule03 / rule14 / rule15 confirmed on the third-party side).

3.3 Network / cloud layer

Three confirmed cloud-side patterns dominate.

Real HTTP business endpoints in shipped assets: **Senssun SmartLink** ships smartlinkdev.senssun.com, 10.8.8.128:8524, and globaltest.senssun.com over HTTP (rule06 confirmed); **FitPro** hardcodes the cleartext WebSocket ws://hiapi.jusonsmart.com:7780/websocket/%s/%s that carries the user id and a token-derived value in the URL path (BR006 / BR013 confirmed); **WeightTrack** ships http://account.withings.com/oauth2_user/authorize2 as the current OAuth authorise entry and http://wbsapi.withings.net/measure on the legacy WithingsActivity path with OAuth token, userid, and

signature handling (BR013 confirmed); **Nutrack** also includes MQTT example defaults using plain tcp:// (kept outside the rule catalogue as non-core).

Fixed-key encrypted uploads on the first-party sync path: **Yuwell uHealth** wraps the SyncService upload payload with the DES("8yu&well") helper described in 3.2.2; DatabaseServiceImpl.getUploadDataArray() → SyncService → the encrypted upload is closed at the code level, so BR091 / BR094 / BR097 stay supported because the actual captured request body was not produced. **BLE Health Plus** wraps its Room database backup with a fixed-IV AES cipher before uploading to Google Drive (R057 / R059 / R060 supported through the same chain).

TLS-bypass clients reaching real cloud surfaces: **xDrip+** ShareRest exchanges Dexcom Share authentication and BG uploads over a trust-all SSLContext (BR009 / BR010 / BR012 confirmed at the code level; the resulting cloud-side leakage stays supported without a packet capture); **Polar Beat** /register/user submits credentials over the same trust-all client (BR009 / BR010 / BR012 confirmed); **bruHealth** StepsUploader sends step and calorie data plus wlapptoken and device-id over its empty-checkServerTrusted client (BR009 / BR010 / BR012 confirmed).

Confirmed first-party health upload chains to HTTPS endpoints: **MyVitale** — UploadMeasures.java:31 builds a Measures object and calls the repository upload; RemoteMeasuresDataSource.java:26 calls measuresApi.postMeasures(...); MeasuresApi.java:37 sets the default base URL to https://vivagym.myvitale.com (R007 confirmed). **Polar Beat** — TrainingSession.java:466 uploads /training-sessions/create; :491 uploads /physical-information; Exercise.java:378/410/418/426/450/458/466 uploads samples, route, RR samples, swimming samples, and transition data (BR091 confirmed). **WeightTrack** — FirebaseUtil / WeightInputActivity / BackupService / util.ad write weight, progress, waist, hip, muscle, heart rate, body fat, water, visceral fat, diary, and other fields to https://weighttrackassistant.firebaseio.com/prd/sync (BR091 confirmed).

Supported first-party cloud chains (static closed, payload not captured): **Rossmax** (DataManager.insertVitalData → FirebaseVitalStore → users/{uid}/measurements/ plus Cloud Functions; R007 / R052 / R059 / R060 supported); **FitPro** (BaseReceiveData.java:1798-1799 saves and immediately calls uploadHeartBlood; HttpHelper.java:456-462 serialises to JSON; CommonService.java:357/365/369/372 sinks uploadHeartBlood / sleep / steps / temperature; UploadDataService.java:58-164 uploads stored history; BR091 / BR094 / BR097 supported); **Bupa Anywhere** (bundle points to services, consumer API, GraphQL, platform graph, telemetry; BR091 / BR094 / BR096 supported); **Nutrack** (NutrackDbOP.java:161-181 serialises changed business tables; NutrackDbHttp.java:670/687-690/694 posts to SYNC.php with dev_id=android_id + Bearer; BR091 / BR092 / BR094 / BR095 supported); **LightBlue** (Retrofit base https://io.adafruit.com/api/v2; P5.a.java:56-66 defines feed list, feed check, feed creation, and feed data POST methods using x-aio-key; characteristicdetail.b.java:779-808 / :964-972 / :1324-1329 stores incoming bytes, checks the UUID-to-Adafruit mapping, and starts the upload coroutine; BR091 / BR097 / BR098 supported); **bruHealth** (StepsUploader path plus health upload APIs and screenshots; BR091 / BR094 / BR095 / BR097 supported); **xDrip+** (UploadChunk.java:51-56 includes Treatments, BloodTests, and BgReadings; TidepoolUploader.java:54,212 uploads those chunks; BR091 / BR095 / BR096 supported).

Third-party telemetry adds a parallel surface: **Rossmax** FireAnalytics logs vital_unit alongside its analytics events; **Polar Beat** enables Facebook and GA enableAdvertisingIdCollection(true); **Bupa Anywhere** wires AppCenter, Braze, Google, Branch, and Sentry; **MyVitale** uses the Omron Firebase-style BigQuery helper. The per-SDK egress payload is in every case supported_hypothesis: the static initialisation and the credentials and identifiers handed to the SDK are visible, but the bytes that leave the device are not.

User-configurable base URL / test environment in production code: **MyVitale** — PreferenceManager.java:36-41 stores measuresServerUrl; AuthPresenter.java:202-203,

UsersListPresenter.java:397-398, and PreferencesActivity.java:125-132 all save it; VitaleAuthService.java:259-261, MeasuresApi.java:174-176, ProfilesApi.java:174-176, and UsersApi.java:218-220 all rebuild request URLs from it (independent engineering finding; kept visible as a trust-boundary problem). **Polar Beat** — TestUIActivity.java:79 writes RemoteServer, :121-128 exposes Production / Preproduction / Nightly / Symbio / Mobile sandbox / Mopsi Test / Test sandbox, :151-156 includes exercise start / stop / status / delete / load / dir buttons (BR060 confirmed for the existence of environment-switching logic inside the production package; external-reachability is deliberately not claimed).

3.4 Multi-layer combined issues

This section keeps only chains that are not visible from any single layer in isolation. The chains below are the ones that earned their place in the report because each component on its own would not have been alarming under any single rule in database_main, and the risk only emerges when two or three components are joined.

(a) **Yuwell HealthManager** — fetal BLE → toString() → release log. A FetalHeartMeasurement object is built from raw fetal-heart audio frames received over BLE. The release-build code path emits this object into the standard Android log pipeline through a default toString() rendering (FetalHeartMeasurement.java:19,26-30 include value, measure time, and local file paths in toString()), on a logger that the same app keeps active in release (FetusHome.java:60). Looking only at the BLE callback would not raise an issue (the parser is correct); looking only at the logger would not raise one either (it is a generic Log.i). The risk only appears once the two are joined: an unencrypted, vendor-private fetal-heart waveform leaves the device-bound BLE channel and lands on a writable log surface inside the app.

(b) **Senssun SmartLink** — external app.zip → UNI Web layer replacement. The Manifest exposes a deep-link entry on the smartlink:// scheme. The embedded UNI WebView layer that drives the business UI is loaded from a .zip archive on the device, and access to the location of that .zip is not gated by an exclusive permission. Either component on its own is a small finding: deep-link entries are common, and zip-driven web layers are common. Together, they form a supply-chain entry into a health app: a process that drops a same-named .zip into the loader path can swap the entire web business layer the next time the user opens the app.

(c) **Senssun SmartLink** — flutter.sp_token across local file, log, and remote JS. The same authentication token reaches three different tiers in the same app. It is persisted in flutter.sp_token; it is then printed at native MainActivity.initData() through Log.i (rule41 confirmed); and the member-points/web.js asset embeds it into a remote sharecn.senssun.com URL builder. Each step is a small finding in its own rule, but the chained exposure of one long-lived credential at a local file, in logcat, and inside a remotely loaded JS surface, is materially worse than any of the three taken alone.

(d) **Yuwell uHealth** — DES sync + cleartext-permitted base + log of decrypted upload. The hardcoded-DES sync wrapper (BR017 confirmed) is not by itself a credential leak. The cleartext-permitted base policy (BR006 confirmed) is not by itself proof of misuse. The log line that prints the decrypted upload JSON together with accessToken and deviceId (BR047 confirmed) is what closes the chain: anyone with logcat-level access on the device can read what the server would have received, including the authentication material, regardless of the DES wrapper or the outer transport.

(e) **BLE Health Plus** — fixed-IV AES + encrypted DB backup + Google Drive destination. The fixed-IV cipher is the same cipher used to wrap the local SQLite backup that the app uploads to the user's Google Drive. The fixed IV alone is a code-quality finding; the third-party backup destination alone is a feature.

The combination means the encrypted backup objects sitting in users' Drive accounts can be decrypted with key material that is shipped inside the public APK.

(f) **Nutrack** — cleartext sync + classic Bluetooth / Wi-Fi sockets + Crashlytics custom keys. **Nutrack's** Crashlytics setCustomKey calls embed account-side identifiers (usuario, android_id, cuenta, licencia). The same app sends sync over a cleartext channel, uses classic Bluetooth and Wi-Fi sockets with unkeyed MD5 integrity, and also sends android_id as dev_id in the SYNC.php query string. Neither the Crashlytics call, nor the cleartext sync, nor the classic Bluetooth path is by itself a high-severity privacy problem; together they create a cross-tier path where an account identifier reaches a third-party crash backend at the same time as the same identifier travels in the clear over the air.

4. Conclusion

Across the fifteen Android BLE health applications reviewed, the most consistent confirmed findings cluster into six categories: cleartext transport at the package-policy level (**FitPro**, **Nutrack**, **Bupa**, **xDrip+**, **Yuwell uHealth**, **Senssun SmartLink**, with **Yuwell HealthManager** as broader posture); hardcoded crypto keys on real product paths (**Yuwell uHealth** DES, **xDrip+** AES backup, **BLE Health Plus** fixed-IV AES, **Nutrack** fixed AES); plain local persistence of credentials and health data (all vendor apps in varying degrees, with **nRF Toolbox** and **LightBlue** still holding at least custom macro storage and Adafruit secret material); release-build logging that mixes sensitive data with identifiers (**MyVitale** health values, **Yuwell HealthManager** fetal paths, **Yuwell uHealth** sync + BLE, **xDrip+** Dexcom credentials, **Senssun SmartLink** token, **FitPro** token + BLE + health, **WeightTrack** OAuth, **bruHealth** sinks); exported components and WebView / JS bridges with controllable inputs (**Yuwell uHealth** WebView, **FitPro** debug / command / upload services, **Bupa** video consultation, **BLE Health Plus** share report, **Yuwell HealthManager** statistics WebViews, **Rossmax** onJsPrompt); and a dense third-party SDK identifier surface (**Bupa** AppCenter / Braze / Branch / Sentry, **FitPro** QQ / Sina / Tencent / Umeng / WeChat, **LightBlue** Mailchimp + Firebase, **Senssun SmartLink** JPush + UMeng, **Yuwell uHealth** JPush / UMeng / Udesk / Youzan, **nRF** Firebase with gating, **Polar Beat** FB + GA ADID).

The most consistent supported_hypothesis findings cluster differently. They concern BLE command injection, replay, and spoofing in apps where no application-layer signature gate is visible between the BLE bytes and the local model (all vendor apps plus **Nutrack's** classic-Bluetooth analogue); pre-consent or undisclosed third-party SDK telemetry in apps where the static SDK initialisation is wired but the actual egress bytes were not captured; and the eventual cloud delivery of credentials and health values whose static client-to-server chain is closed but whose wire bytes were not captured. The two-tier evidence framing is what keeps these distinct from the confirmed set.

The multi-layer chains gathered in section 3.4 illustrate the most important point of the study. Each chain is held together by elements that, taken in isolation, would not have triggered any single rule strongly. A fixed IV is a code-quality issue; a Google Drive destination is a feature; a Log.i call is a debug habit; a deep-link entry is a Manifest convenience. It is the path between them that creates the actual exposure. A review that scans only one layer at a time is structurally unable to see those paths, which is why a uniform cross-layer workflow is the most useful contribution this review can make to the next iteration of the analysis.

References

The full per-rule reference set that backs the rules used in this report is kept in the GitHub repository under `database_main/`, together with the PDFs of each paper and the mapping from paper to rule to keyword.

[1] OWASP Foundation. OWASP Mobile Application Security Verification Standard (MASVS), v2.x. Used as the cross-cutting reference for storage, crypto, and platform-interaction rules.

[2] OWASP Foundation. OWASP Mobile Application Security Testing Guide (MASTG). Used for the per-control checklists that informed the keyword catalogue.

[3] M. Ryan. Bluetooth: With Low Energy comes Low Security. Proceedings of the 7th USENIX Workshop on Offensive Technologies (WOOT), 2013. Foundational reference for BLE link-layer pairing weaknesses.

[4] K. Fawaz, K.-H. Kim, and K. G. Shin. Protecting privacy of BLE device users. Proceedings of the 25th USENIX Security Symposium, 2016. Reference for advertisement-side identifier exposure.

[5] A. P. Felt, M. Finifter, E. Chin, S. Hanna, and D. Wagner. A survey of mobile malware in the wild. Proceedings of the 1st ACM Workshop on Security and Privacy in Smartphones and Mobile Devices (SPSM), 2011. Background for the third-party SDK identifier-collection rules.

[6] L. Onwuzurike and E. De Cristofaro. Danger is My Middle Name: Experimenting with SSL Vulnerabilities in Android Apps. Proceedings of the 8th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec), 2015. Reference for the trust-all TrustManager and HostnameVerifier rules used in 3.2.1.

[7] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez, S. Sundaresan, M. Allman, C. Kreibich, and P. Gill. Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem. Proceedings of NDSS, 2018. Reference for the third-party telemetry observations in 3.3.

[8] R. Stevens, C. Gibler, J. Crussell, J. Erickson, and H. Chen. Investigating User Privacy in Android Ad Libraries. Proceedings of MoST, 2012. Background for the SDK identifier surface in 3.2.8.

The corresponding PDFs and the per-paper rule mapping are available in the repository at `database_main/`.